

ELEG 55903: Programming for Power Electronics

HW9

Tyler Cash

ELEG 4593

HW9

21 April 2026

Abstract:

In this homework assignment, we expand previous assignments and incorporate a Serial Peripheral Interface Analog-to-Digital (SPI ADC) module into our design. The SPI ADC module consists of a SPI interface as well as a Bus Interface to write ADC values to parameters to the SPRAM. Then the full project is controlled using the provided LabView interface via the Serial Port.

Table of Contents:

Abstract:.....	2
List of Figures:	3
Overview:.....	4
Modules:	4
SPI ADC:	4
Conclusion:.....	4
Appendix A (ADC Interface):	6
Appendix B (Bus_Interface_Top):	27
Appendix C (Bus_Interface_tb):.....	37

List of Figures:

FIGURE 1. ADC INTERFACE WAVEFORM	5
FIGURE 2. ADC LABVIEW INTERFACE	5

Overview:

This project was an expansion of our previous homework assignments and midterm. We added a SPI ADC module to our existing design, this allows us to use the LabView interface to control the full project, writing ADC value parameters to the SPRAM. The deliverables for the project are a working simulation, as well as the ADC values for the VPot in the LabVIEW interface. The inputs and outputs for the top level are:

Input: RX, SPI_Dout

Output: TX, LED_1, LED_2, LED_3, LED_4, SPI_clk, SPI_Din, SPI_CS_n

Modules:

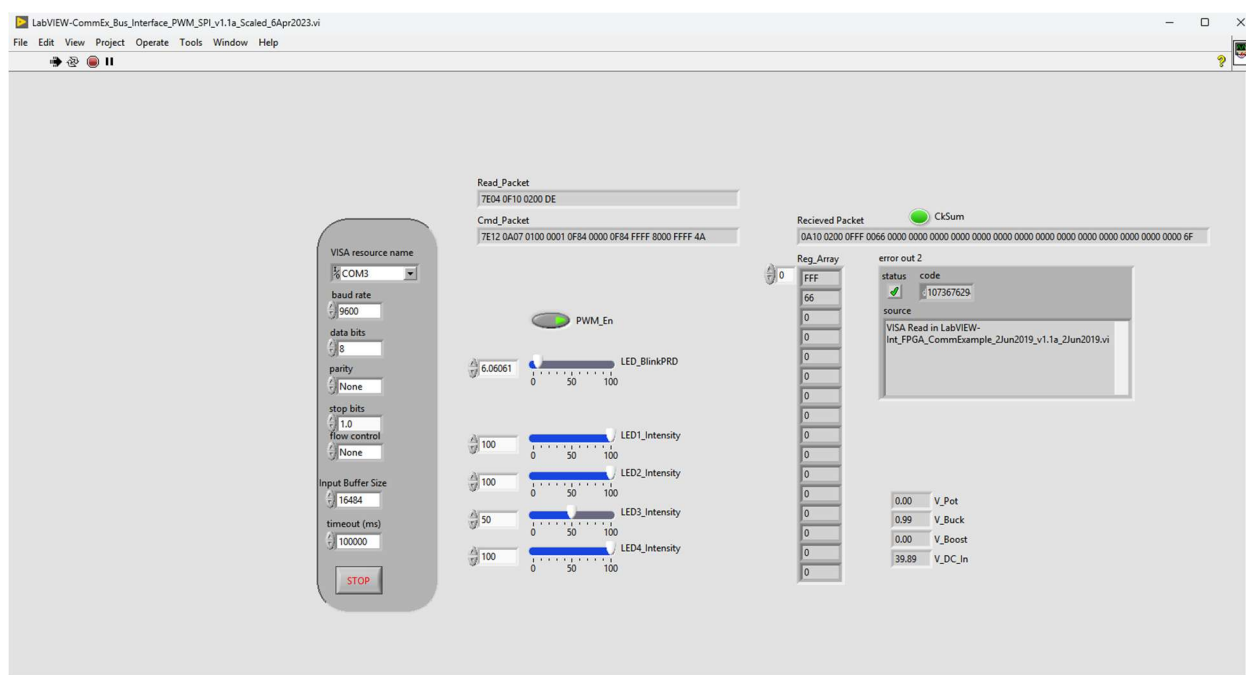
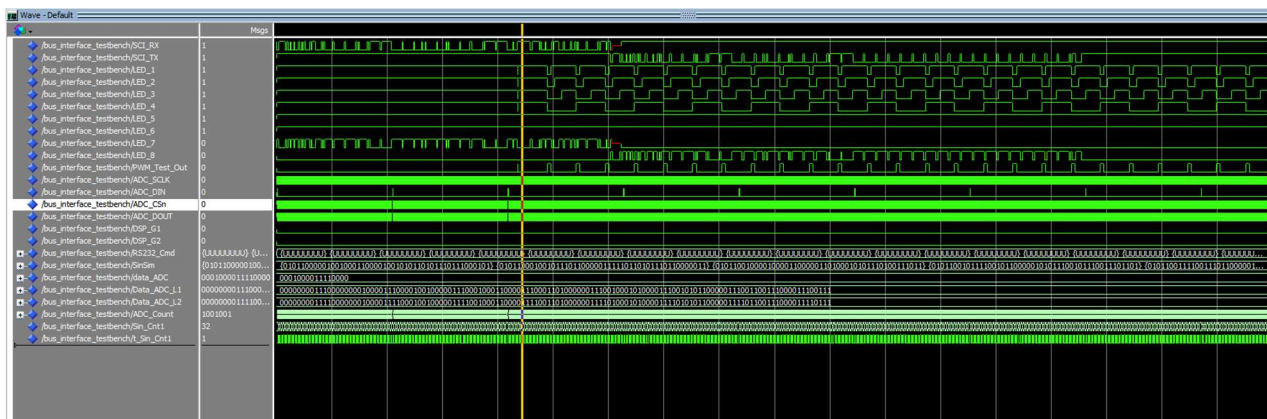
For this assignment, there we build mostly on top of our previous work, only needing to make some changes to the Bus Interface Top and then adding the SPI ADC Interface.

SPI ADC:

This assignment's major implementation was the SPI ADC, which allows the system to read the analog input values from our potentiometer voltage (VPOT) and then convert them to digital data, which is then used to adjust system parameters, like the VBuck, updating the registers and PWM control.

Conclusion:

In conclusion, we properly incorporated the SPI ADC, however I was not able to complete successful implementation in the LabView Interface. The simulation in Figure 1 appears correct, and I was reading values in the LabView Interface but there is still a bug in my code somewhere.



Appendix A (ADC Interface):

```
-----
-- Company: University of Arkansas (NCREPT)
-- Engineer: Tyler Cash
--
-- Create Date: 13Apr2026
-- Last Updated: 22Jan2026
-- Design Name: Bus Interface w/ UART & PWM & SPI
-- Module Name: SPI_ADC_INT - Behavioral
-- Target Devices: LCMX03D-9400HC-6BG256C (MachX03D_BreakoutBrd)
-- Tool versions: Lattice Diamond_x64 Build 3.12.0.240.2
-- Description:
-- This Project was developed to demonstrate a system-level design which incorporates
shared registers and communications.
-- You may use various Serial Interface programs to communicate with this including
the old version of X-CTU (Version 5.2.8.6),
-- Termite, HypterTerminal, or the Custom LabVIEW Interface created for this project
(LabVIEW-CommEx_Bus_Interface_PWM_v1.0a_16Apr2021).
-- A key to remember is that you will need to create packets in Hex-Mode not the
standard ASCII-Mode.
-- Four individual on-board LEDs will be controlled via a simple PWM interface.
-- Each PWM will have its duty cycle set via system registers which may be modified
via the serial interface.
-- A Testbench for this project was created and demonstrates base functionality.
--
---- PinOut:
-- Signal CPLD_Pin Description
-- LED_1 H11 LED1 (Output)
[Onboard LED]
-- LED_2 J13 LED2 (Output)
[Onboard LED]
-- LED_3 J11 LED3 (Output)
[Onboard LED]
-- LED_4 L12 LED4 (Output)
[Onboard LED]
-- LED_5 K11 LED5 (Output)
[Onboard LED]
-- LED_6 L13 LED6 (Output)
[Onboard LED]
-- LED_7 N15 LED7 (Output)
[Onboard LED]
-- LED_8 P16 LED8 (Output)
[Onboard LED]
-- PWM_Test_Out N14 PWM Test (Output)
-- SCI_TX C11 Serial-TX (Output) [USB
Serial Port on Breakout Board]
-- SCI_RX A11 Serial-RX (Input)
[USB Serial Port on Breakout Board]
-- ADC_SCLK E6
-- ADC_DIN C5
-- ADC_CSn C4
-- ADC_DOUT D6
-- DSP_G1 C12
-- DSP_G2 E11
```

```

--
---- Register and Memory Map Information:
-- This section describes the Memory Map used in this project.
-- This design contains a SPRAM Module which is 16 bits wide and 1024 entries deep.
-- Register addresses are from X"0000" to X"03FF".
-- All registers are 16-bits wide.
-- The SPRAM Module is located in the Bus_Master portion of the code.
-- This RAM Module may be accessed externally using Serial Port interface.
-- Reserved for future use.
-- X"0300" - X"03FF"
--
-- LED Configuration Values-
-- Range is X"0100" - X"010A"
--
-- SPI ADC Measurement Values-
-- Range is X"0200" - X"020A"
--
-- Register Map is found as constants in Bus_Interface_Common and shared with all
submodules of this program.
--
---- Serial Settings
-- Baud: 9600
-- Flow Control: None
-- Data Bits: 8
-- Parity: None
-- Stop Bits: 1
--
--
--
---- Serial Communications:
-- In the project we will construct packets to facilitate serial communication
between the device and a computer.
-- Using packets we can implement operational commands and check sums which allow for
easily implementing a user GUI.
-- You may use various Serial Interface programs to communicate with this including
the old version of X-CTU (Version 5.2.8.6),
-- Termite, HypterTerminal, or the Custom LabVIEW Interface created for this project
(LabVIEW-CommEx_Bus_Interface_PWM_v1.0a_16Apr2021)
-- A key to remember is that you will need to create packets in Hex-Mode not the
standard ASCII-Mode.
-- All packets will begin with a Start Delimiter (0x7E) and end with a CheckSum.
-- The CheckSum is calculated by adding up all the bytes of the packet between the
packet length and the checksum then
-- subtracting the sum from 0xFF.
-- We will implement two types of packets for our use.
-- Register Write Command Packet (Used to update registers internal to the CPLD)
-- Register Read Command Packet (Used to read registers internal to the CPLD)
--
---Serial Write Packet (OP_ID = 0x0A)
-- The Register Write Command Packet is used to update registers internal to the
CPLD.
-- The following example breaks down a write request packet.
-- The packet below writes 7 16-bit registers starting at register 0x0100; Values
listed below.

```



```

-- Updated on 27Mar2021 to use a DPRAM module to take advantage of EBRs and maintain
compatibility with other programs.
--
---- Revision 2.1a -
-- Updated for Git Implementation
--
---- Revision 2.0a -
-- Updated for MachX03D and so it could serve as an example for the "Programming for
Power Electronics" Class.
-- Testbed updated to allow for Read and Write Command Verification
-- Minor Updates to documentation and PWMs.
-- Testbench updated to allow for Read and Write Command Verification
--
---- Revision 1.1a -
-- Updated to use Protocols based on Zigbee Implementation
--
-- Revision 1.0b -
-- Updated to use UCB instead of Evaluation Board
--
-- Revision 0.01 -
-- File Created; Basic\Classical Operation Implemented
--
--
-- Additional Comments:
--
--
-----

```

```

Library IEEE;
use IEEE.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;

```

```

entity SPI_ROM is
    port ( rst, clk, D_Sel: in std_logic;
          data: out std_logic_vector(15 downto 0));
end SPI_ROM;

```

architecture behavior of SPI_ROM is

```

begin
    SPI_ROM_behav: process
    begin
        -- X"831F";           -- Command to read ADC Channel 0
        -- X"FFDF";           -- Command to read All ADC Channels
Continuously
        -- X"E3DF";           -- Command to read ADC Channel 0
Continuously
        -- X"E7DF";           -- Command to read ADC Channel 1+2
Continuously
        -- X"FFDF";           -- Command to read All ADC Channels
Continuously
        -- X"871F";           -- Command to read ADC Channel 1
    end process;
end behavior;

```

```

-- X"8B1F";          -- Command to read ADC Channel 2
-- X"8F1F";          -- Command to read ADC Channel 3
-- X"931F";          -- Command to read ADC Channel 4
-- X"971F";          -- Command to read ADC Channel 5
-- X"9B1F";          -- Command to read ADC Channel 6
-- X"9F1F";          -- Command to read ADC Channel 7

wait until clk'event and clk = '1';
if rst='0' then
    data <= (Others =>'0');
else
    if D_Sel = '0' then
        data <= X"FFDF";
    else
        data <= X"FFFF";
    end if;
end if;

end process;
end behavior;
-----END ROM for ADC Commands -----
-----

library IEEE;
use IEEE.std_logic_1164.ALL;
use IEEE.STD_LOGIC_ARITH.all;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use ieee.numeric_std.All;

library work;
use work.Bus_Interface_Common.all;

entity ADC_Int is
    Port (
        clk          : in          STD_LOGIC;
        rst          : in          STD_LOGIC;
        Data         : INOUT std_logic_vector(15 downto 0);
        Addr         : out         std_logic_vector(15 downto 0);
        Xrqst        : out         std_logic;
        XDat         : in          std_logic;
        YDat         : out         std_logic;
        BusRqst      : out         std_logic;
        BusCtrl      : in          std_logic;
        SPI_Sclk     : INOUT std_logic;
        SPI_Din      : INOUT std_logic;
        SPI_CSn      : out         std_logic;
        SPI_Dout     : in          std_logic
    );
end ADC_Int;

architecture Behavioral of ADC_Int is

    ----Constants
    constant Chan          : std_logic_vector(2 downto 0) := "111";
    -- Set maximum sampled ADC

```

```

--constant Delay          :      std_logic_vector(15 downto 0) := X"07FE";
-- Initial ADC delay for Sync
constant Delay            :      std_logic_vector (15 downto 0):=
X"00BA";                -- Initial ADC delay for Sync
constant Offset_Chan      :      std_logic_vector(7 downto 0):= X"07";
-- Delay between samples
constant Offset_Set       :      std_logic_vector(7 downto 0):= X"1E";
-- This is the delay between samples
constant Sample_Size      :      integer := 2;
-- Sample Size for ADC

type state_type is (S0, S1, S2, S3, S4, S5, S6, S7, S8, S9, S10, S11, S12,
S13, S14, S15, S16, S17, S18);
signal CS_ADC_Ctrl, NS_ADC_Ctrl, CS_Bus_Ctrl, NS_Bus_Ctrl: state_type:= S0;

signal SPI_Cnt_INC, SPI_Cnt_rst, Prg_Cnt_INC, Prg_Cnt_rst, Setup_Cnt_INC,
Setup_Cnt_rst:std_logic:='0';
signal Prg_Cnt_Out, Setup_Cnt_Out : std_logic_vector (7 downto 0):= (others =>
'0');
signal SPI_Cnt_Out: std_logic_vector (15 downto 0):= (others => '0');

Signal SPI_ROM1_D_SEL      : std_logic:= '0';
--signal SPI_ROM1_addr: std_logic_vector(2 downto 0);
signal SPI_ROM1_data       : std_logic_vector(15 downto 0):= (others => '0');

signal SPI_PS_ld_D, SPI_PS_sh_D, SPI_PS_rst : std_logic:= '0';
--signal SPI_SP1_Data_In : std_logic_vector (15 downto 0);

signal SPI_SP_ld_D, SPI_SP_rst: STD_LOGIC:='0';
signal SPI_SP1_Data_Out, SPI_SP2_Data_Out :std_logic_vector(15 downto 0):=
(others => '0');

signal SPI_Dout1_Sync : std_logic:='0';
signal SPI_Dout2_Sync : std_logic:='0';

--Signals for ADC Memory
type Memory is array(15 downto 0) of std_logic_vector (15 downto 0);
signal ADC_Mem: Memory;
signal ADC_wea:STD_LOGIC:='0';
signal ADC_Mem_Ave: Memory;
signal ADC_Ave_Ld, ADC_Ave_En : std_logic:='0';
signal ADC_Mem_Temp: Memory;
signal ADC_Temp_rst:std_logic:= '0';
signal Ave_Cnt_INC, Ave_Cnt_rst : std_logic:='0';
signal Ave_Cnt_Out : std_logic_vector(7 downto 0):= (others => '0');
signal ADC_Mem_Ave_M1, ADC_Mem_Ave_M2: Memory;

--Declare signals for Bus Interface
signal Bus_Int1_WE, Bus_Int1_RE, Bus_Int1_Busy: STD_LOGIC:= '0';
signal Bus_Int1_DataIn, Bus_Int1_DataOut, Bus_Int1_AddrIn: std_logic_vector(15
downto 0):= (others => '0');
signal Bus_Cnt_rst, Bus_Cnt_Inc: STD_LOGIC:= '0';

```

```

Signal Bus_Cnt_Out: std_logic_vector(15 downto 0):= (others => '0');
signal Mem_Addr_Cnt_rst, Mem_Addr_Cnt_INC: STD_LOGIC:= '0';
signal Mem_Addr_Cnt_Out: std_logic_vector(7 downto 0):= (others => '0');

--For Clock Divider
signal clk_temp2 : std_logic_vector(3 downto 0):= (others => '0');
signal clk_SPI : std_logic := '0';

--For SPI Setup Loop Counter
signal SPI_CNT_Loop_INC, SPI_CNT_Loop_rst :std_logic:='0';
signal SPI_CNT_Loop_Out : std_logic_vector(15 downto 0):= (others => '0');

--declare STD_Counter component
component Std_Counter is
generic
(
    width : integer    --width of Counter
);
port(INC,rst,clk: in std_logic;
      Count: out std_logic_vector(Width-1 downto 0));
end component;

-- declare SPI_ROM component
component SPI_ROM is
    Port (      clk    : in std_logic;
             rst      : in std_logic;
             D_Sel    : in std_logic;
             data     : out std_logic_vector(15 downto 0)
            );
end component;

-- declare Sreg_PS_16 component
component Sreg_PS_16 is
    Port(  clk      : in std_logic;
          rst       : in std_logic;
          ld_D      : in std_logic;
          sh_D      : in std_logic;
          Data_In    : in std_logic_vector(15 downto 0);
          Data_Out: out std_logic
          );
end component;

-- declare Sreg_SP_16 component
component Sreg_SP_16 is
    Port(  clk      : in std_logic;
          rst       : in std_logic;
          ld_D      : in std_logic;
          Data_In    : in std_logic;
          Data_Out: out std_logic_vector(15 downto 0));

```

```

end component;

--declare Bus Interface
component Bus_Int
  Port(
    clk    :    IN          std_logic;
    rst    :    IN          std_logic;
    DataIn  :    IN          std_logic_vector(15 downto 0);
    DataOut :    OUT        std_logic_vector(15 downto 0);
    AddrIn  :    IN          std_logic_vector(15 downto 0);
    WE      :    IN          std_logic;
    RE      :    IN          std_logic;
    Busy    :    OUT        std_logic;
    Data    :    INOUT      std_logic_vector(15 downto 0);
    Addr    :    OUT        std_logic_vector(15 downto 0);
    Xrqst   :    OUT        std_logic;
    XDat    :    IN          std_logic;
    YDat    :    OUT        std_logic;
    BusRqst :    OUT        std_logic;
    BusCtrl :    IN          std_logic
  );
END component;

begin

  --Connect Signals
  SPI_sclk <= clk_SPI;
  --SPI_Din<=SPI_PS_Data_Out;

  --instantiate SPI_Cnt_16
  SPI_Cnt: STD_Counter
  generic map
  (
    Width => 16
  )
  port map(
    clk => clk_SPI,
    rst => SPI_Cnt_rst,
    INC => SPI_Cnt_Inc,
    Count => SPI_Cnt_Out);

  --instantiate Prg_Cnt_8
  Prg_Cnt: STD_Counter
  generic map
  (
    Width => 8
  )
  port map(
    clk => clk_SPI,
    rst => Prg_Cnt_rst,
    INC => Prg_Cnt_INC,
    Count => Prg_Cnt_Out);

```

```

--instantiate Setup_Cnt_8
Setup_Cnt: STD_Counter
generic map
(
    Width => 8
)
port map(
    clk => clk_SPI,
    rst => Setup_Cnt_rst,
    INC => Setup_Cnt_INC,
    Count => Setup_Cnt_Out);

--instantiate Ave_Cnt_8
Ave_Cnt: STD_Counter
generic map
(
    Width => 8
)
port map(
    clk => clk_SPI,
    rst => Ave_Cnt_rst,
    INC => Ave_Cnt_INC,
    Count => Ave_Cnt_Out);

--instantiate Mem_Cnt
Mem_Cnt: STD_Counter
generic map
(
    Width => 8
)
port map(
    clk => clk,
    rst => Mem_Addr_Cnt_rst,
    INC => Mem_Addr_Cnt_INC,
    Count => Mem_Addr_Cnt_Out);

--instantiate SPI_ROM1_addr
SPI_ROM1: SPI_ROM port map (
    clk => clk_SPI,
    rst => rst,
    D_Sel => SPI_ROM1_D_SEL,
    data => SPI_ROM1_data);

--instantiate SPI_PS
SPI_PS: Sreg_PS_16 port map(
    clk => clk_SPI,
    rst => SPI_PS_rst,
    ld_D => SPI_PS_ld_D,
    sh_D => SPI_PS_sh_D,
    --Data_In => SPI_PS_Data_In,
    Data_In => SPI_ROM1_data,
    Data_Out => SPI_Din);

--instantiate SPI_SP for ADC1
SPI_SP: sreg_SP_16 port map(

```

```

        clk => clk_SPI,
        rst => SPI_SP_rst,
        ld_D => SPI_SP_ld_D,
        Data_In => SPI_Dout1_Sync,
        Data_Out => SPI_SP1_Data_Out);

--Instantiate Bus Interface
Bus_Int1: Bus_Int PORT map(
    clk => clk,
    rst => rst,
    DataIn => Bus_Int1_DataIn,
    DataOut=> Bus_Int1_DataOut,
    AddrIn => Bus_Int1_AddrIn,
    WE => Bus_Int1_WE,
    RE => Bus_Int1_RE,
    Busy => Bus_Int1_Busy,
    Data => Data,
    Addr => Addr,
    Xrqst => Xrqst,
    XDat => XDat,
    YDat => YDat,
    BusRqst => BusRqst,
    BusCtrl => BusCtrl
);

--Instantiate Bus_Cnt_16
Bus_Cnt: STD_Counter
generic map
(
    Width => 16
)
port map(
    clk => clk,
    rst => Bus_Cnt_rst,
    INC => Bus_Cnt_Inc,
    Count => Bus_Cnt_Out);

--instantiate Bus_Cnt_16
SPI_Loop_Cnt: Std_Counter
generic map
(
    Width => 16
)
port map(
    clk => clk,
    rst => SPI_CNT_Loop_rst,
    INC => SPI_CNT_Loop_INC,
    Count => SPI_CNT_Loop_Out);

----Process for storing sampled data into ADC Memory
ADC_Mem_W: Process
begin
    wait until clk_SPI'event and clk_SPI = '1';

```

```

if rst = '0' then
    ADC_Mem(0) <= (others => '0');
    ADC_Mem(1) <= (others => '0');
    ADC_Mem(2) <= (others => '0');
    ADC_Mem(3) <= (others => '0');
    ADC_Mem(4) <= (others => '0');
    ADC_Mem(5) <= (others => '0');
    ADC_Mem(6) <= (others => '0');
    ADC_Mem(7) <= (others => '0');
    ADC_Mem(8) <= (others => '0');
    ADC_Mem(9) <= (others => '0');
    ADC_Mem(10) <= (others => '0');
    ADC_Mem(11) <= (others => '0');
    ADC_Mem(12) <= (others => '0');
    ADC_Mem(13) <= (others => '0');
    ADC_Mem(14) <= (others => '0');
    ADC_Mem(15) <= (others => '0');

    elsif ADC_wea = '1' then
        if SPI_SP1_Data_Out(14 downto 12) = "000" then ADC_Mem(0)
<= X"0" & SPI_SP1_Data_Out(11 downto 0); end if;
        if SPI_SP1_Data_Out(14 downto 12) = "001" then ADC_Mem(1)
<= X"0" & SPI_SP1_Data_Out(11 downto 0); end if;
        if SPI_SP1_Data_Out(14 downto 12) = "010" then ADC_Mem(2)
<= X"0" & SPI_SP1_Data_Out(11 downto 0); end if;
        if SPI_SP1_Data_Out(14 downto 12) = "011" then ADC_Mem(3)
<= X"0" & SPI_SP1_Data_Out(11 downto 0); end if;
        if SPI_SP1_Data_Out(14 downto 12) = "100" then ADC_Mem(4)
<= X"0" & SPI_SP1_Data_Out(11 downto 0); end if;
        if SPI_SP1_Data_Out(14 downto 12) = "101" then ADC_Mem(5)
<= X"0" & SPI_SP1_Data_Out(11 downto 0); end if;
        if SPI_SP1_Data_Out(14 downto 12) = "110" then ADC_Mem(6)
<= X"0" & SPI_SP1_Data_Out(11 downto 0); end if;
        if SPI_SP1_Data_Out(14 downto 12) = "111" then ADC_Mem(7)
<= X"0" & SPI_SP1_Data_Out(11 downto 0); end if;

    end if;
end Process;
----End Registers

----Process for averaging sampled data
Sample_Ave: Process
begin
    wait until clk_SPI'event and clk_SPI = '1';
    if (rst = '0' or ADC_Temp_rst = '0') then
        ADC_Mem_Temp(0) <= (others => '0');
        ADC_Mem_Temp(1) <= (others => '0');
        ADC_Mem_Temp(2) <= (others => '0');
        ADC_Mem_Temp(3) <= (others => '0');
        ADC_Mem_Temp(4) <= (others => '0');
        ADC_Mem_Temp(5) <= (others => '0');
        ADC_Mem_Temp(6) <= (others => '0');
        ADC_Mem_Temp(7) <= (others => '0');
        ADC_Mem_Temp(8) <= (others => '0');

```



```

        ADC_Mem_Temp(9) <= (others => '0');
        ADC_Mem_Temp(10) <= (others => '0');
        ADC_Mem_Temp(11) <= (others => '0');
        ADC_Mem_Temp(12) <= (others => '0');
        ADC_Mem_Temp(13) <= (others => '0');
        ADC_Mem_Temp(14) <= (others => '0');
        ADC_Mem_Temp(15) <= (others => '0');

    elsif ADC_Ave_En = '1' then

        ADC_Mem_Temp(0) <= ADC_Mem(0) + ADC_Mem_Temp(0);
        ADC_Mem_Temp(1) <= ADC_Mem(1) + ADC_Mem_Temp(1);
        ADC_Mem_Temp(2) <= ADC_Mem(2) + ADC_Mem_Temp(2);
        ADC_Mem_Temp(3) <= ADC_Mem(3) + ADC_Mem_Temp(3);
        ADC_Mem_Temp(4) <= ADC_Mem(4) + ADC_Mem_Temp(4);
        ADC_Mem_Temp(5) <= ADC_Mem(5) + ADC_Mem_Temp(5);
        ADC_Mem_Temp(6) <= ADC_Mem(6) + ADC_Mem_Temp(6);
        ADC_Mem_Temp(7) <= ADC_Mem(7) + ADC_Mem_Temp(7);
        ADC_Mem_Temp(8) <= ADC_Mem(8) + ADC_Mem_Temp(8);
        ADC_Mem_Temp(9) <= ADC_Mem(9) + ADC_Mem_Temp(9);
        ADC_Mem_Temp(10) <= ADC_Mem(10) + ADC_Mem_Temp(10);
        ADC_Mem_Temp(11) <= ADC_Mem(11) + ADC_Mem_Temp(11);
        ADC_Mem_Temp(12) <= ADC_Mem(12) + ADC_Mem_Temp(12);
        ADC_Mem_Temp(13) <= ADC_Mem(13) + ADC_Mem_Temp(13);
        ADC_Mem_Temp(14) <= ADC_Mem(14) + ADC_Mem_Temp(14);
        ADC_Mem_Temp(15) <= ADC_Mem(15) + ADC_Mem_Temp(15);

    end if;
end Process;
-- End Registers

----Process for storing averaged sample data
ADC_Mem_Ave_Reg: Process
begin
    wait until clk_SPI'event and clk_SPI = '1';
    if (rst = '0') then
        ADC_Mem_Ave(0) <= (others => '0');
        ADC_Mem_Ave(1) <= (others => '0');
        ADC_Mem_Ave(2) <= (others => '0');
        ADC_Mem_Ave(3) <= (others => '0');
        ADC_Mem_Ave(4) <= (others => '0');
        ADC_Mem_Ave(5) <= (others => '0');
        ADC_Mem_Ave(6) <= (others => '0');
        ADC_Mem_Ave(7) <= (others => '0');
        ADC_Mem_Ave(8) <= (others => '0');
        ADC_Mem_Ave(9) <= (others => '0');
        ADC_Mem_Ave(10) <= (others => '0');
        ADC_Mem_Ave(11) <= (others => '0');
        ADC_Mem_Ave(12) <= (others => '0');
        ADC_Mem_Ave(13) <= (others => '0');
        ADC_Mem_Ave(14) <= (others => '0');
        ADC_Mem_Ave(15) <= (others => '0');

    elsif ADC_Ave_Ld = '1' then

```

```

        ADC_Mem_Ave(0) <= b"0" & ADC_Mem_Temp(0)(15 downto 1);
        ADC_Mem_Ave(1) <= b"0" & ADC_Mem_Temp(1)(15 downto 1);
        ADC_Mem_Ave(2) <= b"0" & ADC_Mem_Temp(2)(15 downto 1);
        ADC_Mem_Ave(3) <= b"0" & ADC_Mem_Temp(3)(15 downto 1);
        ADC_Mem_Ave(4) <= b"0" & ADC_Mem_Temp(4)(15 downto 1);
        ADC_Mem_Ave(5) <= b"0" & ADC_Mem_Temp(5)(15 downto 1);
        ADC_Mem_Ave(6) <= b"0" & ADC_Mem_Temp(6)(15 downto 1);
        ADC_Mem_Ave(7) <= b"0" & ADC_Mem_Temp(7)(15 downto 1);
        ADC_Mem_Ave(8) <= b"0" & ADC_Mem_Temp(8)(15 downto 1);
        ADC_Mem_Ave(9) <= b"0" & ADC_Mem_Temp(9)(15 downto 1);
        ADC_Mem_Ave(10) <= b"0" & ADC_Mem_Temp(10)(15 downto 1);
        ADC_Mem_Ave(11) <= b"0" & ADC_Mem_Temp(11)(15 downto 1);
        ADC_Mem_Ave(12) <= b"0" & ADC_Mem_Temp(12)(15 downto 1);
        ADC_Mem_Ave(13) <= b"0" & ADC_Mem_Temp(13)(15 downto 1);
        ADC_Mem_Ave(14) <= b"0" & ADC_Mem_Temp(14)(15 downto 1);
        ADC_Mem_Ave(15) <= b"0" & ADC_Mem_Temp(15)(15 downto 1);

    end if;
end Process;
----End Registers

---Process for crossing clock domains from SPI to clk to prevent metastability
ADC_Mem_Ave_Reg_Meta: Process
begin
    wait until clk'event and clk = '1';
    if (rst = '0') then
        ADC_Mem_Ave_M1(0) <= (others => '0');
        ADC_Mem_Ave_M1(1) <= (others => '0');
        ADC_Mem_Ave_M1(2) <= (others => '0');
        ADC_Mem_Ave_M1(3) <= (others => '0');
        ADC_Mem_Ave_M1(4) <= (others => '0');
        ADC_Mem_Ave_M1(5) <= (others => '0');
        ADC_Mem_Ave_M1(6) <= (others => '0');
        ADC_Mem_Ave_M1(7) <= (others => '0');
        ADC_Mem_Ave_M1(8) <= (others => '0');
        ADC_Mem_Ave_M1(9) <= (others => '0');
        ADC_Mem_Ave_M1(10) <= (others => '0');
        ADC_Mem_Ave_M1(11) <= (others => '0');
        ADC_Mem_Ave_M1(12) <= (others => '0');
        ADC_Mem_Ave_M1(13) <= (others => '0');
        ADC_Mem_Ave_M1(14) <= (others => '0');
        ADC_Mem_Ave_M1(15) <= (others => '0');
        ADC_Mem_Ave_M2(0) <= (others => '0');
        ADC_Mem_Ave_M2(1) <= (others => '0');
        ADC_Mem_Ave_M2(2) <= (others => '0');
        ADC_Mem_Ave_M2(3) <= (others => '0');
        ADC_Mem_Ave_M2(4) <= (others => '0');
        ADC_Mem_Ave_M2(5) <= (others => '0');
        ADC_Mem_Ave_M2(6) <= (others => '0');
        ADC_Mem_Ave_M2(7) <= (others => '0');
        ADC_Mem_Ave_M2(8) <= (others => '0');
        ADC_Mem_Ave_M2(9) <= (others => '0');
        ADC_Mem_Ave_M2(10) <= (others => '0');
    end if;
end Process;

```

```

        ADC_Mem_Ave_M2(11) <= (others => '0');
        ADC_Mem_Ave_M2(12) <= (others => '0');
        ADC_Mem_Ave_M2(13) <= (others => '0');
        ADC_Mem_Ave_M2(14) <= (others => '0');
        ADC_Mem_Ave_M2(15) <= (others => '0');

    else

        ADC_Mem_Ave_M1(0) <= ADC_Mem_Ave(0);
        ADC_Mem_Ave_M1(1) <= ADC_Mem_Ave(1);
        ADC_Mem_Ave_M1(2) <= ADC_Mem_Ave(2);
        ADC_Mem_Ave_M1(3) <= ADC_Mem_Ave(3);
        ADC_Mem_Ave_M1(4) <= ADC_Mem_Ave(4);
        ADC_Mem_Ave_M1(5) <= ADC_Mem_Ave(5);
        ADC_Mem_Ave_M1(6) <= ADC_Mem_Ave(6);
        ADC_Mem_Ave_M1(7) <= ADC_Mem_Ave(7);
        ADC_Mem_Ave_M1(8) <= ADC_Mem_Ave(8);
        ADC_Mem_Ave_M1(9) <= ADC_Mem_Ave(9);
        ADC_Mem_Ave_M1(10) <= ADC_Mem_Ave(10);
        ADC_Mem_Ave_M1(11) <= ADC_Mem_Ave(11);
        ADC_Mem_Ave_M1(12) <= ADC_Mem_Ave(12);
        ADC_Mem_Ave_M1(13) <= ADC_Mem_Ave(13);
        ADC_Mem_Ave_M1(14) <= ADC_Mem_Ave(14);
        ADC_Mem_Ave_M1(15) <= ADC_Mem_Ave(15);
        ADC_Mem_Ave_M2(0) <= ADC_Mem_Ave_M1(0);
        ADC_Mem_Ave_M2(1) <= ADC_Mem_Ave_M1(1);
        ADC_Mem_Ave_M2(2) <= ADC_Mem_Ave_M1(2);
        ADC_Mem_Ave_M2(3) <= ADC_Mem_Ave_M1(3);
        ADC_Mem_Ave_M2(4) <= ADC_Mem_Ave_M1(4);
        ADC_Mem_Ave_M2(5) <= ADC_Mem_Ave_M1(5);
        ADC_Mem_Ave_M2(6) <= ADC_Mem_Ave_M1(6);
        ADC_Mem_Ave_M2(7) <= ADC_Mem_Ave_M1(7);
        ADC_Mem_Ave_M2(8) <= ADC_Mem_Ave_M1(8);
        ADC_Mem_Ave_M2(9) <= ADC_Mem_Ave_M1(9);
        ADC_Mem_Ave_M2(10) <= ADC_Mem_Ave_M1(10);
        ADC_Mem_Ave_M2(11) <= ADC_Mem_Ave_M1(11);
        ADC_Mem_Ave_M2(12) <= ADC_Mem_Ave_M1(12);
        ADC_Mem_Ave_M2(13) <= ADC_Mem_Ave_M1(13);
        ADC_Mem_Ave_M2(14) <= ADC_Mem_Ave_M1(14);
        ADC_Mem_Ave_M2(15) <= ADC_Mem_Ave_M1(15);

    end if;
end process;
---- End Registers

----Next state logic for ADC Control Signals
ADC_Ctrl: process(SPI_Dout1_Sync, SPI_Dout2_Sync, NS_ADC_Ctrl, CS_ADC_Ctrl,
SPI_Cnt_Out, Prg_Cnt_Out, Setup_CNT_Out, Ave_Cnt_Out, SPI_CNT_Loop_Out)
begin

    ----- Define Default States to prevent Latches
    SPI_ROM1_D_SEL   <=      '0';
    SPI_PS_rst       <=      '1';
    SPI_PS_ld_D      <=      '0';
    SPI_PS_sh_D      <=      '0';

```

```

SPI_Cnt_rst    <=      '1';
SPI_Cnt_INC    <=      '0';
Setup_Cnt_rst    <=      '1';
Setup_Cnt_INC<=      '0';
Prg_Cnt_rst    <=      '1';
Prg_Cnt_INC      <=      '0';

--SPI_Din      <=      '0';
SPI_CS_n      <=      '1';
SPI_SP_rst      <=      '1';
SPI_SP_ld_D     <=      '0';

--ADC_Mem
ADC_wea        <=      '0';
ADC_Ave_En      <=      '0';
Ave_Cnt_rst     <=      '1';
Ave_Cnt_INC     <=      '0';
ADC_Temp_rst    <=      '1';
ADC_Ave_Ld      <=      '0';

--For Setup Loop
SPI_CNT_Loop_rst<=      '1';
SPI_Cnt_Loop_INC <=      '0';

```

```

case CS_ADC_Ctrl is
when S0 =>
    SPI_Cnt_rst    <= '0';
    Setup_Cnt_rst  <= '0';
    Prg_Cnt_rst    <= '0';
    SPI_SP_rst     <= '0';
    SPI_PS_rst     <= '0';
    ADC_Temp_rst   <= '0';
    Ave_Cnt_rst    <= '0';
    SPI_CNT_Loop_rst<= '0';
    NS_ADC_Ctrl    <= S1;

    --Delay for Alignment
    when S1 =>
        if(SPI_Cnt_Out < Delay) then
            SPI_Cnt_Inc <= '1';
            NS_ADC_Ctrl <= S1;
        else
            SPI_Cnt_rst<= '0';
            NS_ADC_Ctrl <= S2;
        end if;

    --Begin Dmy Load
    when S2 =>
        SPI_ROM1_D_SEL <= '1';
        SPI_PS_ld_D <= '1';
        SPI_Cnt_rst <= '0';

```

```

        NS_ADC_Ctrl1 <= S3;

when S3 =>
    SPI_ROM1_D_SEL <= '1';
    SPI_PS_ld_D <= '1';
    NS_ADC_Ctrl1 <= S4;

when S4 =>
    NS_ADC_Ctrl1 <= S5;

when S5 =>
    SPI_CS_n <= '0';
    if (SPI_Cnt_Out < 15) then
        SPI_Cnt_INC <= '1';
        NS_ADC_Ctrl1 <= S5;
    elsif (Setup_Cnt_Out < 1) then
        Setup_Cnt_INC <= '1';
        SPI_ROM1_D_SEL <= '1';
        NS_ADC_Ctrl1 <= S2;
    else
        SPI_Cnt_rst <= '0';
        NS_ADC_Ctrl1 <= S6;

    end if;
--End Dummy Load

----Begin ADC Loop

--Delay for beginning of Loop
when S6 =>
    if(SPI_CNT_Out < 10) then
        SPI_CNT_Inc <= '1';
        NS_ADC_Ctrl1 <= S6;
    else
        NS_ADC_Ctrl1 <= S7;
    end if;

--Begin Write Command
when S7 =>
    SPI_Cnt_rst <= '0';
    SPI_SP_rst <= '0';
    Prg_Cnt_rst <= '0';
    SPI_PS_ld_D <= '1';
    NS_ADC_Ctrl1 <= S8;

when S8 =>
    SPI_PS_sh_D <= '1';
    --SPI_SP_ld_D <= '1';
    NS_ADC_Ctrl1 <= S9;

when S9 =>
    SPI_PS_sh_D <= '1';
    SPI_Cnt_INC <= '1';
    SPI_CS_n <= '0';

```

```

        if (SPI_Cnt_Out < 15) then
            NS_ADC_Ctrl <= S9;
        else
            NS_ADC_Ctrl <= S10;
        end if;
--End Write Command

when S10 =>
    SPI_Cnt_rst <= '0';
    SPI_SP_rst <= '0';
    NS_ADC_Ctrl <= S11;

--Begin Read Command
when S11 =>
    NS_ADC_Ctrl <= S12;

when S12 =>
    SPI_Cnt_INC <= '1';
    SPI_CS_n <= '0';
    SPI_SP_ld_D <= '1';
    if (SPI_CNT_Out < 15) then
        NS_ADC_Ctrl <= S12;
    else
        NS_ADC_Ctrl <= S13;
    end if;

when S13 =>
    SPI_Cnt_rst <= '0';
    --ADC_wea <= '1';
    NS_ADC_Ctrl <= S14;

when s14 =>
    ADC_wea <= '1';
    NS_ADC_Ctrl <= S15;

when s15 =>
    if(SPI_Cnt_Out < Offset_Chan) then
        SPI_CNT_Inc <= '1';
        NS_ADC_Ctrl <= S15;
    elsif(Prg_Cnt_Out < Chan ) then
        Prg_Cnt_INC <= '1';
        NS_ADC_Ctrl <= S10;
    else
        SPI_Cnt_rst <= '0';
        Prg_Cnt_rst <= '0';
        Setup_Cnt_rst <= '0';
        NS_ADC_Ctrl <= S16;
    end if;

when S16 =>
    if (Ave_Cnt_Out < (Sample_Size - 1)) then
        NS_ADC_Ctrl <= S18;
    else
        NS_ADC_Ctrl <= S17;
    end if;

```

```

        ADC_Ave_En <= '1';
        Ave_Cnt_INC <= '1';

    when S17 =>
        ADC_Ave_Ld <= '1';
        ADC_Temp_rst <= '0';
        Ave_Cnt_rst <= '0';
        SPI_Cnt_Inc <= '1';
        NS_ADC_Ctrl <= S18;

    when S18 =>
        if(SPI_Cnt_Out < Offset_Set) then
            SPI_CNT_Inc <= '1';
            NS_ADC_Ctrl <= S18;
        else
            SPI_Cnt_rst <= '0';
            Prg_Cnt_rst <= '0';
            if (SPI_CNT_Loop_Out < 1000) then
                SPI_CNT_Loop_INC <= '1';
                NS_ADC_Ctrl <= S10;
            else
                NS_ADC_Ctrl <= S0;
            end if;
        end if;

    when others =>
        NS_ADC_Ctrl <= S0;

    end case;
end process;
----Next State Logic for ADC Control Signals

```

```

----Next State Logic for Bus Interface
NSL_Bus: process(CS_Bus_Ctrl, Bus_Cnt_Out, Bus_Int1_Busy, Mem_Addr_Cnt_Out,
ADC_Mem_Ave_M2)
Begin

    ----Default States to remove Latches
    Bus_Int1_AddrIn <= (others => '0');
    Bus_Int1_RE <= '0';
    Bus_Int1_DataIn <= (others => '0');
    Bus_Int1_WE <= '0';
    Bus_Cnt_rst <= '1';
    Bus_Cnt_Inc <= '0';

```

```

Mem_Addr_Cnt_rst <= '1';
Mem_Addr_Cnt_INC <= '0';

case CS_Bus_Ctrl is
  when S0 =>
    Bus_Cnt_rst <= '0';
    Mem_Addr_Cnt_rst <= '0';
    NS_Bus_Ctrl <= S1;

    --Write Measurement Data to the Bus
    when S1 =>
      if(Bus_Cnt_Out < 4002) then
        NS_Bus_Ctrl <= S1;
      else
        NS_Bus_Ctrl <= S2;
      end if;
      Bus_Cnt_Inc <= '1';

    when S2 =>
      if(Bus_Int1_Busy = '1') then
        NS_Bus_Ctrl <= S2;
      else
        NS_Bus_Ctrl <= S3;
      end if;

    when S3 =>
      if(Mem_Addr_Cnt_Out < 15) then
        NS_Bus_Ctrl <= S2;
      else
        NS_Bus_Ctrl <= S0;
      end if;

      --ADC map
      if(Mem_Addr_Cnt_Out = 0) then Bus_Int1_AddrIn <=
Addr_ADC0; end if;
      if(Mem_Addr_Cnt_Out = 1) then Bus_Int1_AddrIn <=
Addr_ADC1; end if;
      if(Mem_Addr_Cnt_Out = 2) then Bus_Int1_AddrIn <=
Addr_ADC2; end if;
      if(Mem_Addr_Cnt_Out = 3) then Bus_Int1_AddrIn <=
Addr_ADC3; end if;
      if(Mem_Addr_Cnt_Out = 4) then Bus_Int1_AddrIn <=
Addr_ADC4; end if;
      if(Mem_Addr_Cnt_Out = 5) then Bus_Int1_AddrIn <=
Addr_ADC5; end if;
      if(Mem_Addr_Cnt_Out = 6) then Bus_Int1_AddrIn <=
Addr_ADC6; end if;
      if(Mem_Addr_Cnt_Out = 7) then Bus_Int1_AddrIn <=
Addr_ADC7; end if;
      if(Mem_Addr_Cnt_Out = 8) then Bus_Int1_AddrIn <=
Addr_ADC8; end if;
      if(Mem_Addr_Cnt_Out = 9) then Bus_Int1_AddrIn <=
Addr_ADC9; end if;

```



```

Addr_ADC10; end if;
Addr_ADC11; end if;
Addr_ADC12; end if;
Addr_ADC13; end if;
Addr_ADC14; end if;
Addr_ADC15; end if;

```

```

Bus_Int1_DataIn <=
ADC_Mem_Ave_M2(conv_integer(Mem_Addr_Cnt_Out(3 downto 0))); -- set data
Mem_Addr_Cnt_INC <= '1';
Bus_Int1_WE <= '1';

```

```

when others =>
    NS_Bus_Ctrl <= S0;

```

```

end case;
end process;
----END Next State Logic for Bus Interface

```

```

-- Sync Process for FSMs
sync_ADC: Process
Begin
    wait until clk_SPI'event and clk_SPI = '1';
    if rst = '0' then
        CS_ADC_Ctrl <= S0;
    else
        CS_ADC_Ctrl <= NS_ADC_Ctrl;
    end if;
end process;

```

```

-- Sync process for FSMs
sync_Bus: Process
Begin
    wait until clk'event and clk = '1';
    if rst = '0' then
        CS_Bus_Ctrl <= S0;
    else
        CS_Bus_Ctrl <= NS_Bus_Ctrl;
    end if;
end process;

```

```

--- Synch for Dout Signals
sync_SPI: Process
Begin
    wait until clk_SPI'event and clk_SPI = '0';
    if rst = '0' then
        SPI_Dout1_Sync <= '0';
    else
        SPI_Dout1_Sync <= SPI_Dout;
    end if;
end process;

-- Clock Divider for SPI_Sclk
Clk_Div_Top: Process
Begin
    wait until clk'event and clk = '1';
    clk_temp2 <= clk_temp2+1;
    --clk_SPI <= clk_temp2(3);
    clk_SPI <= clk_temp2(2);
    --clk_SPI <= not(clk_SPI);
end process;

end Behavioral;

```

Appendix B (Bus Interface Top):

-- Company: University of Arkansas (NCREPT)
-- Engineer: Tyler Cash
--
-- Create Date: 25March2026
-- Last Updated: 7AprilMar2026
-- Design Name: Bus Interface w/ UART
-- Module Name: BusInterface_UART - Behavioral
-- Project Name: BusInterface_UART
-- Target Devices: LCMXO3D-9400HC-6BG256C (MachXO3D_BreakoutBrd)
-- Tool versions: Lattice Diamond_x64 Build 3.12.0.240.2
-- Description:
-- This Project was developed to demonstrate a system-level design which incorporates shared registers and communications.
-- You may use various Serial Interface programs to communicate with this including the old version of X-CTU (Version 5.2.8.6),
-- Termite, HypterTerminal, or the Custom LabVIEW Interface created for this project (LabVIEW-CommEx_Bus_Interface_PWM_v1.0a_16Apr2021).
-- A key to remember is that you will need to create packets in Hex-Mode not the standard ASCII-Mode.
-- A Testbench for this project was created and demonstrates base functionality.
--
---- PinOut:
-- Signal CPLD_Pin Description
-- SCI_TX C11 Serial-TX (Output) [USB
Serial Port on Breakout Board]
-- SCI_RX A11 Serial-RX (Input) [USB
Serial Port on Breakout Board]
--
---- Register and Memory Map Information:
-- This section describes the Memory Map used in this project.
-- This design contains a SPRAM Module which is 16 bits wide and 1024 entries deep.
-- Register addresses are from X"0000" to X"03FF".
-- All registers are 16-bits wide.
-- The SPRAM Module is located in the Bus_Master portion of the code.
-- This RAM Module may be accessed externally using Serial Port interface.
-- Reserved for future use.
-- X"0300" - X"03FF"
-- Register Map is found as constants in Bus_Interface_Common and shared with all submodules of this program.
--
---- Serial Settings
-- Baud: 9600
-- Flow Control: None
-- Data Bits: 8
-- Parity: None
-- Stop Bits: 1

```

--
--
--
---- Serial Communications:
-- In the project we will construct packets to facilitate serial communication between the device and a
computer.
-- Using packets we can implement operational commands and check sums which allow for easily
implementing a user GUI.
-- You may use various Serial Interface programs to communicate with this including the old version of
X-CTU (Version 5.2.8.6),
-- Terminate, HyperTerminal, or the Custom LabVIEW Interface created for this project (LabVIEW-
CommEx_Bus_Interface_PWM_v1.0a_16Apr2021)
-- A key to remember is that you will need to create packets in Hex-Mode not the standard ASCII-Mode.
-- All packets will begin with a Start Delimiter (0x7E) and end with a CheckSum.
-- The CheckSum is calculated by adding up all the bytes of the packet between the packet length and
the checksum then
-- subtracting the sum from 0xFF.
-- We will implement two types of packets for our use.
-- Register Write Command Packet (Used to update registers internal to the CPLD)
-- Register Read Command Packet (Used to read registers internal to the CPLD)
--
---Serial Write Packet (OP_ID = 0x0A)
-- The Register Write Command Packet is used to update registers internal to the CPLD.
-- The following example breaks down a write request packet.
-- The packet below writes 7 16-bit registers starting at register 0x0100; Values listed below.
-- PWM_Enable    => 0x0001 (1=Enable; 0=Disable)
-- LED_BlinkFreq  => 0xBFFF [75%]  (BFFF/FFFF) (%)
-- LED_OnTime     => 0x6000 [50%]  (6000/BFFF) (%)
-- LED1_Intensity => 0x2000 [12.5%] (2000/FFFF) (%)
-- LED2_Intensity => 0x4000 [25%]  (4000/FFFF) (%)
-- LED3_Intensity => 0x8000 [50%]  (8000/FFFF) (%)
-- LED4_Intensity => 0xFFFF [100%] (FFFF/FFFF) (%)
--
-- Start Delimiter| Pkt Len |Op ID| Register_Cnt |Start Address |Register Data (16bit x Register_Cnt) |
ChkSum
-- 0x7E      | 0x12 |0x0A|0x07      |0x0100      |0x0001BFFF6000200040008000FFFF      | 0xF0
-- 0x7E120A0701000001BFFF6000200040008000FFFFF0 results in all LEDs being set to the above
parameters.
-- 0x7E120A0701000001BFFF60000000000000000000CE results in all LED Intensities being set to 0%.
--
--- Serial Read Packet (OP_ID = 0x0F)
-- The Register Read Command Packet is used to read registers internal to the CPLD.
-- The following example breaks down a read request packet.
-- The packet below reads 16 16-bit registers starting at register 0x0100.
-- Start Delimiter| Pkt Len |Op ID| Register_Cnt |Start Address | ChkSum
-- 0x7E      | 0x04 |0x0F|0x10      |0x0100      | 0xDF
-- 0x7E040F100100DF
--

```



```
--
-- Additional Comments:
--
--10
-----
```

```
Library IEEE;
use IEEE.std_logic_1164.all;
use ieee.numeric_std.all;
use IEEE.std_logic_arith.all;
```

```
library machxo3d;
use machxo3d.all;
```

```
library work;
use work.Bus_Interface_Common.all;
```

```
entity Bus_Interface_Top is
    Port (
```

```
        ---- RS232 Communication
        SCI_RX : in STD_LOGIC;
        SCI_TX : inout STD_LOGIC;

        LED_1 : out STD_LOGIC;
        LED_2 : out STD_LOGIC;
        LED_3 : out STD_LOGIC;
        LED_4 : out STD_LOGIC;
        LED_5 : out STD_LOGIC;
        LED_6 : out STD_LOGIC;
        LED_7 : out STD_LOGIC;
        LED_8 : out STD_LOGIC;
        PWM_Test_Out : out STD_LOGIC;
        ADC_SCLK: INOUT std_logic;
        ADC_DIN: INOUT std_logic;
        ADC_CSn: OUT STD_LOGIC;
        ADC_Dout: IN STD_LOGIC;
        DSP_G1: out STD_LOGIC;
        DSP_G2: out STD_LOGIC
    );
```

```
end Bus_Interface_Top;
```

```
architecture Behavioral of Bus_Interface_Top is
```

```
    Component OSCJ
    GENERIC (NOM_FREQ: string := "8.31");
    PORT ( STDBY : IN std_logic;
          OSC : OUT std_logic;
          SEDSTDBY : OUT std_logic
```

```

    );
END COMPONENT;

-- Declare PLL
Component PLLSerial
PORT (
    ClkI: in std_logic;
    ClkOP: out std_logic;
    Lock: out std_logic
);
END COMPONENT;

-- Declare Bus Master
COMPONENT Bus_Master
PORT (
    clk : IN std_logic;
    rst : IN std_logic;
    Data : INOUT STD_LOGIC_VECTOR(15 downto 0);
    Addr : IN STD_LOGIC_VECTOR(15 downto 0);
    Xrqst : IN std_logic;
    XDat : OUT std_logic;
    YDat : IN std_logic;
    BusRqst : IN std_logic_vector(9 downto 0);
    BusCtrl : OUT std_logic_vector(9 downto 0)
);
END Component;

COMPONENT RS232_Usr_Int
generic(
    Baud    : integer;
    clk_in  : integer
);
Port (
    clk : in STD_LOGIC;
    rst : in STD_LOGIC;
    rs232_rcv : in STD_LOGIC;
    rs232_xmt : out STD_LOGIC;
    Data      : INOUT std_logic_vector(15 downto 0);
    Addr      : OUT   std_logic_vector(15 downto 0);
    Xrqst     : OUT   std_logic;
    XDat      : IN    std_logic;
    YDat      : OUT   std_logic;
    BusRqst   : OUT   std_logic;

```

```

        BusCtrl :      IN      std_logic
    );
end      Component;

-- Declare LED_Ctrl
COMPONENT LED_Ctrl is
Port (
    clk          :      in STD_LOGIC;
    rst          :      in STD_LOGIC;
    Data   :      INOUT std_logic_vector(15 downto 0);
    Addr   :      OUT   std_logic_vector(15 downto 0);
    Xrqst  :      OUT std_logic;
    XDat   :      IN std_logic;
    YDat   :      OUT std_logic;
    BusRqst:      OUT   std_logic;
    BusCtrl :      IN std_logic;
    LED_En :      out std_logic;
    LED1_Out:      out STD_LOGIC;
    LED2_Out:      out STD_LOGIC;
    LED3_Out:      out STD_LOGIC;
    LED4_Out:      out STD_LOGIC;
    LED5_Out:      out STD_LOGIC;
    LED6_Out:      out STD_LOGIC;
    LED7_Out:      out STD_LOGIC;
    LED8_Out:      out STD_LOGIC
);
END COMPONENT;

```

```

--Declare ADC_Int
Component ADC_Int is
PORT (
    clk : in STD_LOGIC;
    rst : in STD_LOGIC;
    Data : inout std_logic_vector(15 downto 0);
    Addr: out std_logic_vector(15 downto 0);
    Xrqst: out std_logic;
    XDat: in STD_LOGIC;
    YDat: out std_logic;
    BusRqst: out STD_LOGIC;
    BusCtrl: in STD_LOGIC;
    SPI_Sclk: inout STD_LOGIC;
    SPI_Din: inout STD_LOGIC;
    SPI_CSn: out STD_LOGIC;
    SPI_Dout: in STD_LOGIC
);
END COMPONENT;

```



```

-- Declare STD_Counter Component
component STD_Counter is
GENERIC
(
    Width : integer--width of Counters
);
Port (INC, rst, clk: in std_logic;
      Count: out std_logic_vector(Width -1 downto 0)
);
end component;

--signals
-- Declare Signals for Bus Interface
signal Bus_Int1_WE, Bus_Int1_RE, Bus_Int1_Busy: STD_LOGIC:= '0';
signal Bus_Int1_DataIn, Bus_Int1_DataOut, Bus_Int1_AddrIn: STD_LOGIC_VECTOR(15 downto
0):= (others => '0');

-- Inputs
signal Addr : std_logic_vector(15 downto 0):= (others => '0');
signal Xrqst : std_logic := '0';
signal YDat : std_logic := '0';
signal BusRqst : STD_LOGIC_Vector(9 downto 0):= (others => '0');
signal Data : STD_LOGIC_Vector(15 downto 0):= (others => '0');
signal XDat : STD_LOGIC := '0';
signal BusCtrl : STD_LOGIC_Vector(9 downto 0):= (others => '0');

-- internal clock
signal OSC_Stdbby, OSC_Out, OSC_SEDSTDBY, clk: std_logic := '0';

-- Reset
signal PLL_Lock, System_rst: std_logic := '0';
signal Reset_Cnt_INC, Reset_Cnt_rst: std_logic := '0';
signal Reset_Cnt_out: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');

--Misc
signal LED_En : std_logic := '0';
-- For inverting LED Outputs
signal LED_1n, LED_2n, LED_3n, LED_4n, LED_5n, LED_6n, LED_7n, LED_8n: STD_LOGIC:= '0';

begin

-- Instantiate Internal Oscillator
Int_OSC: OSCJ Port MAP (
    STDBY => OSC_Stdbby,
    OSC => OSC_Out,
    SEDSTDBY => OSC_SEDSTDBY

```

```

    );

--Instantiate PLL
PLL_1: PLLSerial PORT MAP (
    ClkI => OSC_Out,
    ClkOP => clk,
    Lock => PLL_Lock
);

-- Instantaite Bus_Master
BM: Bus_Master Port Map (
    clk => clk,
    rst => System_rst,
    Data => Data,
    Addr => Addr,
    Xrqst => Xrqst,
    XDat => XDat,
    YDat => YDat,
    BusRqst => BusRqst,
    BusCtrl => BusCtrl
);

-- Instantaite RS232_Usr_Int
RS232_Usr: RS232_Usr_Int
Generic MAP
(
    Baud => 9600,
    Clk_in => Clk_Freq -- Input clk
)
PORT MAP (
    clk => clk,
    rst => System_rst,
    rs232_rcv => SCI_RX,
    rs232_xmt => SCI_TX,
    Data => Data,
    Addr => Addr,
    Xrqst => Xrqst,
    XDat => XDat,
    YDat => YDat,
    BusRqst => BusRqst(2),
    BusCtrl => BusCtrl(2)
);

LED_Ctrl1: LED_Ctrl PORT MAP (
    clk => clk,
    rst => System_rst,
    Data => Data,
    Addr => Addr,

```

```

Xrqst => Xrqst,
XDat => XDat,
YDat => YDat,
BusRqst => BusRqst(0),
BusCtrl => BusCtrl(0),
LED_En => LED_En,
LED1_Out => LED_1n,
LED2_Out => LED_2n,
LED3_Out => LED_3n,
LED4_Out => LED_4n,
LED5_Out => LED_5n,
LED6_Out => LED_6n,
LED7_Out => LED_7n,
LED8_Out => LED_8n
);

--Instantiate ADC_Int
ADC_Int1: ADC_Int PORT MAP (
    clk => clk,
    rst => System_rst,
    SPI_Sclk => ADC_SCLK,
    SPI_Din => ADC_DIN,
    SPI_CSn => ADC_CSn,
    SPI_Dout => ADC_DOUT,
    Data => Data,
    Addr => Addr,
    Xrqst => Xrqst,
    XDat => XDat,
    YDat => YDat,
    BusRqst => BusRqst(1),
    BusCtrl => BusCtrl(1)
);

--Instantiate Reset_Cnt_8
Reset_Cnt: STD_Counter
generic MAP
(
    Width => 8
)
port map(
    clk => OSC_Out,
    rst => Reset_Cnt_rst,
    INC => Reset_Cnt_INC,
    Count => Reset_Cnt_out
);

-- Oscillator
OSC_Stdbby <= '0';

```

```

-- Tie unused ports to '0'
BusRqst(9 downto 3) <= (others => '0');
DSP_G1 <= '0';
DSP_G2 <= '0';

-- Reset Block1
Reset_Blkl: Process
begin
    wait until OSC_Out'event and OSC_Out = '1';
    if(PLL_Lock = '0') then
        Reset_Cnt_rst <= '0'; -- active low
    else
        Reset_Cnt_rst <= '1';
    end if;
end process;

-- Reset Block
Reset_Blkl: process
begin
    wait until OSC_Out'event and OSC_Out = '1';
    if(Reset_Cnt_out < X"7F") then
        System_rst <= '0';
        Reset_Cnt_Inc <= '1';
    else
        System_rst <= '1';
        Reset_Cnt_Inc <= '0';
    end if;
end process;

-- LED Invert due to Active Low Configuration on Dev Board
LED_Invert: process(LED_1n, LED_2n, LED_3n, LED_4n, LED_5n, LED_6n, LED_7n, LED_8n,
SCI_RX, SCI_TX)
begin
    LED_8 <= not(SCI_TX);
    LED_7 <= not(SCI_RX);
    LED_6 <= not(LED_6n);
    LED_5 <= not(LED_5n);
    LED_4 <= not(LED_4n);
    LED_3 <= not(LED_3n);
    LED_2 <= not(LED_2n);
    LED_1 <= not(LED_1n);
    PWM_Test_Out <= LED_1n;

end process;

```

end Behavioral;

Appendix C (Bus Interface tb):

-- Company: University of Arkansas (NCREPT)
-- Engineer: Tyler Cash
--
-- Create Date: 25March2026
-- Last Updated: 7AprilMar2026
-- Design Name: Bus Interface w/ UART
-- Module Name: BusInterface_UART - Behavioral
-- Project Name: BusInterface_UART
-- Target Devices: LCMXO3D-9400HC-6BG256C (MachXO3D_BreakoutBrd)
-- Tool versions: Lattice Diamond_x64 Build 3.12.0.240.2
-- Description:
-- This Project was developed to demonstrate a system-level design which incorporates shared registers and communications.
-- You may use various Serial Interface programs to communicate with this including the old version of X-CTU (Version 5.2.8.6),
-- Termite, HypterTerminal, or the Custom LabVIEW Interface created for this project (LabVIEW-CommEx_Bus_Interface_PWM_v1.0a_16Apr2021).
-- A key to remember is that you will need to create packets in Hex-Mode not the standard ASCII-Mode.
-- A Testbench for this project was created and demonstrates base functionality.
--
---- PinOut:
-- Signal CPLD_Pin Description
-- SCI_TX C11 Serial-TX (Output)[USB Serial Port on Breakout Board]
-- SCI_RX A11 Serial-RX (Input) [USB Serial Port on Breakout Board]
--
---- Register and Memory Map Information:
-- This section describes the Memory Map used in this project.
-- This design contains a SPRAM Module which is 16 bits wide and 1024 entries deep.
-- Register addresses are from X"0000" to X"03FF".
-- All registers are 16-bits wide.
-- The SPRAM Module is located in the Bus_Master portion of the code.
-- This RAM Module may be accessed externally using Serial Port interface.
-- Reserved for future use.
-- X"0300" - X"03FF"
-- Register Map is found as constants in Bus_Interface_Common and shared with all submodules of this program.
--
---- Serial Settings
-- Baud: 9600
-- Flow Control: None
-- Data Bits: 8
-- Parity: None
-- Stop Bits: 1
--

```

--
--
--- Serial Communications:
-- In the project we will construct packets to facilitate serial communication between the device and a computer.
-- Using packets we can implement operational commands and check sums which allow for easily implementing a
user GUI.
-- You may use various Serial Interface programs to communicate with this including the old version of X-CTU
(Version 5.2.8.6),
-- Termite, HypterTerminal, or the Custom LabVIEW Interface created for this project (LabVIEW-
CommEx_Bus_Interface_PWM_v1.0a_16Apr2021)
-- A key to remember is that you will need to create packets in Hex-Mode not the standard ASCII-Mode.
-- All packets will begin with a Start Delimiter (0x7E) and end with a CheckSum.
-- The CheckSum is calculated by adding up all the bytes of the packet between the packet length and the
checksum then
-- subtracting the sum from 0xFF.
-- We will implement two types of packets for our use.
-- Register Write Command Packet (Used to update registers internal to the CPLD)
-- Register Read Command Packet (Used to read registers internal to the CPLD)
--
---Serial Write Packet (OP_ID = 0x0A)
-- The Register Write Command Packet is used to update registers internal to the CPLD.
-- The following example breaks down a write request packet.
-- The packet below writes 7 16-bit registers starting at register 0x0100; Values listed below.
-- PWM_Enable    => 0x0001 (1=Enable; 0=Disable)
-- LED_BlinkFreq  => 0xBFFF [75%] (BFFF/FFFF) (%)
-- LED_OnTime     => 0x6000 [50%] (6000/BFFF) (%)
-- LED1_Intensity => 0x2000 [12.5%] (2000/FFFF) (%)
-- LED2_Intensity => 0x4000 [25%] (4000/FFFF) (%)
-- LED3_Intensity => 0x8000 [50%] (8000/FFFF) (%)
-- LED4_Intensity => 0xFFFF [100%] (FFFF/FFFF) (%)
--
-- Start Delimiter| Pkt Len | Op ID| Register_Cnt | Start Address | Register Data (16bit x Register_Cnt) | ChkSum
-- 0x7E          | 0x12 | 0x0A | 0x07      | 0x0100      | 0x0001BFFF6000200040008000FFFF      | 0xF0
-- 0x7E120A0701000001BFFF6000200040008000FFFFF0 results in all LEDs being set to the above parameters.
-- 0x7E120A0701000001BFFF60000000000000000000CE results in all LED Intensities being set to 0%.
--
--- Serial Read Packet (OP_ID = 0x0F)
-- The Register Read Command Packet is used to read registers internal to the CPLD.
-- The following example breaks down a read request packet.
-- The packet below reads 16 16-bit registers starting at register 0x0100.
-- Start Delimiter| Pkt Len | Op ID| Register_Cnt | Start Address | ChkSum
-- 0x7E          | 0x04 | 0x0F | 0x10      | 0x0100      | 0xDF
-- 0x7E040F100100DF
--
-- The above command results in a write command being sent from the CPLD which contains data from 16 16-bit
registers starting at address 0x0100.
-- An example response from the CPLD is shown below:
-- 0x7E240A1001000001BFFF6000200040008000FFFF00000000000000000000000000000000E7
-- Start Delimiter| Pkt Len | Op ID| Register_Cnt | Start Address | Register Data (16bit x Register_Cnt)
| ChkSum
-- 0x7E          | 0x24 | 0x0A | 0x10      | 0x0100
| 0x0001BFFF6000200040008000FFFF000000000000000000000000000000000000 | 0xE7
--

```

```

--
--
--
--
-----
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use ieee.numeric_std.all;

ENTITY Bus_Interface_TestBench IS
END Bus_Interface_TestBench;

ARCHITECTURE behavior OF Bus_Interface_TestBench IS

    COMPONENT Bus_Interface_Top
    PORT(
        SCI_RX : IN std_logic;
        SCI_TX : INOUT std_logic;
        LED_1 : OUT std_logic;
        LED_2 : OUT std_logic;
        LED_3 : OUT std_logic;
        LED_4 : OUT std_logic;
        LED_5 : OUT std_logic;
        LED_6 : OUT std_logic;
        LED_7 : OUT std_logic;
        LED_8 : OUT std_logic;
        PWM_Test_Out : OUT std_logic;
        ADC_SCLK : INOUT std_logic;
        ADC_DIN : INOUT std_logic;
        ADC_CSn : OUT std_logic;
        ADC_DOUT : IN std_logic;
        DSP_G1 : out STD_LOGIC;
        DSP_G2 : out STD_LOGIC
    );
    END COMPONENT;

    SIGNAL SCI_RX : std_logic;
    SIGNAL SCI_TX : std_logic;
    SIGNAL LED_1 : std_logic;
    SIGNAL LED_2 : std_logic;
    SIGNAL LED_3 : std_logic;
    SIGNAL LED_4 : std_logic;
    SIGNAL LED_5 : std_logic;
    SIGNAL LED_6 : std_logic;
    SIGNAL LED_7 : std_logic;
    SIGNAL LED_8 : std_logic;
    SIGNAL PWM_Test_Out : std_logic;
    SIGNAL ADC_SCLK : std_logic;
    SIGNAL ADC_DIN : std_logic;

```

```

SIGNAL ADC_CSn : std_logic;
SIGNAL ADC_DOUT : std_logic;
SIGNAL DSP_G1 : std_logic;
SIGNAL DSP_G2 : std_logic;

-- Clock period definitions
constant clk_period : time := 20 ns;
--constant read_Time: time :=8680 ns; --for 115,200 Baud
constant read_Time: time :=104100 ns; --for 9,600 Baud

-- System Clk and Reset(Not Needed Here)
--signal SYCLK : std_logic;
--signal RESETn : std_logic;

-- Memory Array
type Memory is array (255 downto 0) of STD_LOGIC_VECTOR (7 downto 0);
signal RS232_Cmd: Memory;

----ADC Data for SPI
type Memory_36 is array (101 downto 0) of STD_LOGIC_VECTOR (47 downto 0);
signal SinSim: Memory_36:=
(X"5824612B5DC5",X"589760FB5D83",X"590860D15D3B",X"597960AE5CED",X"59E760925C9B",X"5A54607D5C44",
X"5ABE606F5BE8",X"5B2560685B88",X"5B8860685B25",X"5BE8606F5ABE",X"5C44607D5A54",X"5C9B609259E7",
X"5CED60AE5979",X"5D3B60D15908",X"5D8360FB5897",X"5DC5612B5824",X"5E02616157B2",X"5E38619E573F",
X"5E6861E056CC",X"5E926228565B",X"5EB5627655EA",X"5ED162C8557C",X"5EE6631F550F",X"5EF4637B54A5",
X"5EFB63DB543E",X"5EFB643E53DB",X"5EF464A5537B",X"5EE6650F531F",X"5ED1657C52C8",X"5EB565EA5276",X"
5E92665B5228",X"5E6866CC51E0",X"5E38673F519E",X"5E0267B25161",X"5DC56824512B",X"5D83689750FB",X"
5D3B690850D1",X"5CED697950AE",X"5C9B69E75092",X"5C446A54507D",X"5BE86ABE506F",X"5B886B255068",X"
5B256B885068",X"5ABE6BE8506F",X"5A546C44507D",X"59E76C9B5092",X"59796CED50AE",X"59086D3B50D1",X"
58976D8350FB",X"58246DC5512B",X"57B26E025161",X"573F6E38519E",X"56CC6E6851E0",X"565B6E925228",X"5
5EA6EB55276",X"557C6ED152C8",X"550F6EE6531F",X"54A56EF4537B",X"543E6EFB53DB",X"53DB6EFB543E",X"53
7B6EF454A5",X"531F6EE6550F",X"52C86ED1557C",X"52766EB555EA",X"52286E92565B",X"51E06E6856CC",X"519
E6E38573F",X"51616E0257B1",X"512B6DC55824",X"50FB6D835897",X"50D16D3B5908",X"50AE6CED5979",X"509
26C9B59E7",X"507D6C445A54",X"506F6BE85ABE",X"50686B885B25",X"50686B255B88",X"506F6ABE5BE8",X"507
D6A545C44",X"509269E75C9B",X"50AE69795CED",X"50D169085D3B",X"50FB68975D83",X"512B68245DC5",X"51
6167B15E02",X"519E673F5E38",X"51E066CC5E68",X"5228665B5E92",X"527665EA5EB5",X"52C8657C5ED1",X"531
F650F5EE6",X"537B64A55EF4",X"53DB643E5EFB",X"543E63DB5EFB",X"54A5637B5EF4",X"550F631F5EE6",X"557C
62C85ED1",X"55EA62765EB5",X"565B62285E92",X"56CC61E05E68",X"573F619E5E38",X"573F619E5E38");

--ADC Simulator Data
signal data_ADC : std_logic_vector(15 downto 0):= b"0001000011110000";
signal Data_ADC_L1: std_logic_vector(127 downto 0):= X"00E010E120E230E340E450E560E670E7";
signal Data_ADC_L2: std_logic_vector(127 downto 0):= X"00F010F120F230F340F450F560F670F7";
signal ADC_Count : std_logic_vector(6 downto 0):=b"1111110";
signal Sin_Cnt1: integer:=0;
signal t_Sin_Cnt1: std_logic:= '0';

```


BEGIN

-- Please check and add your generic clause manually

 ut: Bus_Interface_Top PORT MAP(

 SCI_RX => SCI_RX,

 SCI_TX => SCI_TX,

 LED_1 => LED_1,

 LED_2 => LED_2,

 LED_3 => LED_3,

 LED_4 => LED_4,

 LED_5 => LED_5,

 LED_6 => LED_6,

 LED_7 => LED_7,

 LED_8 => LED_8,

 PWM_Test_Out => PWM_Test_Out,

 ADC_SCLK => ADC_SCLK,

 ADC_DIN => ADC_DIN,

 ADC_CSn => ADC_CSn,

 ADC_DOUT => ADC_Dout,

 DSP_G1 => DSP_G1,

 DSP_G2 => DSP_G2

);

---- Example Serial Commands

--- Set Registers

-- 7E12 0A07 0100 0001 FFFF 0000 2000 4000 8000 FFFF 10

---Read Registers

-- 7E04 0F10 0100 DF

----Define Command Memory

--Test Write

RS232_Cmd(0) <= X"7E";

--Start Deliminator

RS232_Cmd(1) <= X"12";

--Pkt Length

RS232_Cmd(2) <= X"0A";

--Cmd (Read)

RS232_Cmd(3) <= X"07";

--Register Count

RS232_Cmd(4) <= X"01";

--Start Address High

RS232_Cmd(5) <= X"00";

--Start Address Low

RS232_Cmd(6) <= X"00";

--LED Enable High

RS232_Cmd(7) <= X"01";

--LED Enable Low

RS232_Cmd(8) <= X"FF";

--Blink Period High

RS232_Cmd(9) <= X"FF";

--Blink Period Low

RS232_Cmd(10) <= X"00";

--LED On Time High

RS232_Cmd(11) <= X"00";

--LED On Time Low

RS232_Cmd(12) <= X"20";

--LED 1 Intensity High

RS232_Cmd(13) <= X"00";

--LED 1 Intensity Low

RS232_Cmd(14) <= X"40";

--LED 2 Intensity High

RS232_Cmd(15) <= X"00";

--LED 2 Intensity Low

RS232_Cmd(16) <= X"80";

--LED 3 Intensity High

RS232_Cmd(17) <= X"00";

--LED 3 Intensity Low

RS232_Cmd(18) <= X"FF";

--LED 4 Intensity High

```

RS232_Cmd(19) <= X"FF";      --LED 4 Intensity Low
RS232_Cmd(20) <= X"10";      --Check Sum

```

```

--Test Read
RS232_Cmd(30) <= X"7E";      --Start Deliminator
RS232_Cmd(31) <= X"04";      --Pkt Length
RS232_Cmd(32) <= X"0F";      --Cmd (Read)
RS232_Cmd(33) <= X"10";      --Register Count
RS232_Cmd(34) <= X"01";      --Start Address High
RS232_Cmd(35) <= X"00";      --Start Address Low
RS232_Cmd(36) <= X"DF";      --Check Sum

```

-- Stimulus process

stim_proc: process

begin

-- initialize serial ports to idle state

SCI_RX <= '1';

---- hold reset state for 100 ns.

--RESETn <= '0';

--wait for clk_period*100;

--RESETn <= '1';

--wait for clk_period*100;

---- insert stimulus here

wait for clk_period*100;

for j in 0 to 20 loop

SCI_RX <= '0';

--Send Start Bit

wait for read_Time;

for i in 0 to 7 loop

SCI_RX <= RS232_Cmd(0+j)(i);

wait for read_Time;

end loop;

SCI_RX <= '1';

--Send Stop Bit

wait for read_Time;

end loop;

wait for read_Time*10;

for j in 0 to 7 loop

SCI_RX <= '0';

--Send Start Bit

wait for read_Time;

for i in 0 to 7 loop

SCI_RX <= RS232_Cmd(30+j)(i);

wait for read_Time;

end loop;

SCI_RX <= '1';

--Send Stop Bit

wait for read_Time;

end loop;

```
wait; -- will wait forever
```

```
END PROCESS;
```

```
-- ADC Sim
ADC_SIM1 :process
begin
    wait until ADC_SCLK'event and ADC_SCLK = '1';
    if (ADC_CSn = '0') then
        if((ADC_Count >= 32) and (ADC_Count <= 47)) then
            ADC_DOUT <= SinSim(Sin_Cnt1)(32+(CONV_INTEGER(ADC_Count)-32));

        elsif((ADC_Count >= 16) and ( ADC_Count <= 31)) then
            ADC_DOUT <= SinSim(Sin_Cnt1)(16+CONV_INTEGER(ADC_Count)-16);

        else
            ADC_DOUT <= Data_ADC_L1(CONV_INTEGER(ADC_Count));
        end if;

        if(ADC_Count = 1) then
            if(Sin_Cnt1 < 101) then
                if(t_Sin_Cnt1 = '1') then
                    Sin_Cnt1 <= Sin_Cnt1+1;
                    t_Sin_Cnt1 <= '0';
                else
                    t_Sin_Cnt1 <= '1';
                end if;
            else
                Sin_Cnt1 <= 0;
            end if;
        end if;

        ADC_Count <= ADC_Count - 1;

    else
        ADC_DOUT <= '1';
        ADC_Count <= ADC_Count;
    end if;
end process;
```

```
END;
```